

TÀI LIỆU QUẢN TRỊ DỰ ÁN: AI CARO ENGINE

1. TỔNG QUAN DỰ ÁN

1.1. Mục tiêu dự án

Xây dựng một hệ thống AI chơi cờ Caro trên bàn cờ 15x15, có khả năng đánh với người chơi bằng thuật toán **Minimax** kết hợp **Alpha-Beta Pruning**, đồng thời hỗ trợ đo lường hiệu năng thực nghiệm để phục vụ báo cáo và bảo vệ trước Hội đồng.

1.2. Định vị sản phẩm

Đây là một **game Caro 15x15 thiên về trình diễn AI**, không nhằm mô phỏng đầy đủ mọi biến thể luật Caro truyền thống. Trọng tâm của dự án là:

- độ đúng của thuật toán tìm kiếm
- hiệu quả cắt tỉa Alpha-Beta
- chất lượng hàm lượng giá Heuristic
- khả năng trình bày trực quan bằng UI và benchmark

1.3. Triết lý triển khai

Dự án tuân theo nguyên tắc:

- đơn giản hóa luật chơi
- tối ưu hóa lỗi AI
- giới hạn phạm vi để hoàn thành chắc chắn
- mọi thành phần phải phục vụ trực tiếp cho demo và báo cáo

2. PHẠM VI DỰ ÁN VÀ BỘ LUẬT LỖI

2.1. Phạm vi cố định

Toàn bộ dự án chỉ triển khai trong phạm vi sau:

- bàn cờ cố định kích thước **15x15**
- chơi giữa **người và AI**
- AI sử dụng **Minimax + Alpha-Beta Pruning**
- giao diện chạy trên trình duyệt cục bộ
- không thêm tính năng ngoài phạm vi như:
 - chơi online

- database
- tài khoản người dùng
- chat
- xếp hạng

2.2. Luật thắng

Người chơi thắng khi có **từ 5 quân liên tiếp trở lên** theo một trong bốn hướng:

- ngang
- dọc
- chéo chính
- chéo phụ

2.3. Luật đơn giản hóa

Dự án áp dụng bộ luật đơn giản hóa như sau:

- không xét chặn 2 đầu
- overline vẫn tính thắng
- không có luật cấm nước
- không có Renju rule
- không áp dụng luật Caro Việt Nam

2.4. Điều kiện hòa

Ván cờ được xác định là **hòa** khi:

- tổng số ô đã đánh bằng **225**
- và không bên nào thỏa điều kiện thắng

3. QUY TRÌNH KIỂM TRA TRẠNG THÁI VÁN CỜ

3.1. Nguyên tắc

Sau mỗi nước đi, hệ thống phải kiểm tra trạng thái theo đúng trình tự sau.

3.2. Trình tự chuẩn

Bước 1: Kiểm tra thắng

Từ đúng tọa độ quân cờ vừa đặt, quét cục bộ theo 4 hướng:

- ngang
- dọc
- chéo chính
- chéo phụ

Ở mỗi hướng:

- đếm số quân cùng màu về một phía
- đếm số quân cùng màu về phía ngược lại
- cộng với quân hiện tại

Nếu tổng số quân trên bất kỳ một trục nào ≥ 5 thì:

- kết thúc ván cờ ngay
- người vừa đánh là người thắng

Bước 2: Kiểm tra hòa

Nếu bước 1 không xác định thắng, kiểm tra số lượng ô đã được đánh:

- nếu số ô đã đánh = 225
- thì kết thúc ván cờ với kết quả hòa

Bước 3: Đổi lượt

Nếu không thắng và không hòa thì đổi lượt cho phía còn lại.

3.3. Yêu cầu bắt buộc

- tuyệt đối không quét toàn bàn cờ sau mỗi lượt
- chỉ được kiểm tra cục bộ từ ô vừa đánh
- đây là điều kiện bắt buộc để đảm bảo hiệu năng runtime ổn định

4. KIẾN TRÚC KỸ THUẬT

4.1. Công nghệ sử dụng

- **AI Core:** C++
- **Frontend/UI:** HTML + JavaScript + Canvas hoặc DOM
- **Tích hợp:** WebAssembly thông qua Emscripten
- **Đa luồng giao diện:** Web Worker để tránh block Main Thread

4.2. Kiến trúc tổng quát

Hệ thống gồm 4 lớp chính:

1. Game Engine

- quản lý trạng thái bàn cờ 15x15
- kiểm tra nước đi hợp lệ
- kiểm tra thắng/hòa
- quản lý lượt chơi

2. AI Core

- Minimax
- Alpha-Beta Pruning
- Candidate Move Generation
- Evaluation Function
- Iterative Deepening

3. Frontend/UI

- hiển thị bàn cờ
- nhận thao tác người chơi
- hiển thị kết quả AI
- hiển thị chỉ số thời gian thực

4. Logging & Benchmark

- lưu số node đã duyệt
- thời gian xử lý
- score heuristic
- depth đạt được
- phục vụ benchmark và báo cáo

5. TỐI ƯU KHÔNG GIAN TÌM KIẾM

5.1. Nguyên tắc cốt lõi

AI **không được phép** duyệt toàn bộ 225 ô trên bàn cờ để tìm nước đi.

5.2. Candidate Moves

AI chỉ xét các ô trống thỏa mãn điều kiện:

- là ô trống
- và có ít nhất một quân bất kỳ đang tồn tại trong vùng lân cận **Chebyshev distance ≤ 2**

Điều này có nghĩa là:

- AI chỉ nhìn vào vùng đang có giao tranh
- các ô quá xa khỏi khu vực đang chơi bị loại bỏ

5.3. Xử lý nước đi đầu tiên

Nếu bàn cờ hoàn toàn trống, Candidate Moves mặc định chỉ gồm:

- ô trung tâm (7,7)

Đây là mặc định tham khảo để AI có nước mở đầu hợp lý và giảm số lượng nhánh không cần thiết.

5.4. Ý nghĩa học thuật

Đây là một trong những tối ưu quan trọng nhất của dự án vì giúp:

- giảm branching factor
- tăng hiệu quả Alpha-Beta
- đạt thời gian phản hồi tốt hơn
- giữ độ sâu tìm kiếm ở mức thực tế

6. KIẾN TRÚC AI VÀ FALLBACK AN TOÀN

6.1. Thuật toán lõi

AI sử dụng:

- Minimax
- Alpha-Beta Pruning
- Iterative Deepening
- Evaluation Function dựa trên các mẫu chuỗi quân

6.2. Timeout

Mỗi lượt AI được giới hạn:

- `timeout_ms = 2000ms`

6.3. Iterative Deepening

AI không nhảy thẳng vào độ sâu tối đa, mà phải tìm kiếm lần lượt:

- depth 1
- depth 2
- depth 3
- ...
- đến `max_depth` hoặc đến khi hết thời gian

6.4. Cơ chế fallback khi timeout

- khi hoàn thành trọn vẹn một depth, AI phải lưu lại nước đi tốt nhất của depth đó
- nếu bị hết thời gian trong lúc duyệt depth tiếp theo:
 - ngắt tìm kiếm
 - trả về nước đi tốt nhất của depth hoàn chỉnh gần nhất
- không được trả về `null`
- không được làm crash worker hoặc UI

6.5. Fallback tích hợp WASM

Nếu sau 3 ngày đầu sprint, việc tích hợp WASM không đạt yêu cầu, thì áp dụng phương án dự phòng.

Tiêu chí được coi là WASM fail:

- không biên dịch được mã C++ sang WASM
- UI không gọi được hàm AI ổn định
- worker crash lặp lại nhiều lần không xác định được nguyên nhân
- giao tiếp JS-WASM không ổn định, gây lỗi runtime nghiêm trọng

Quy trình quyết định fallback:

- Thành viên 2 báo cáo tình trạng thất bại kỹ thuật
- trưởng nhóm hoặc người phụ trách điều phối ra quyết định chính thức chuyển hướng
- phương án dự phòng là:
 - bọc lỗi C++ bằng Python Flask hoặc FastAPI

- chạy local backend nhẹ
- gọi qua API cục bộ

7. GIAO THỨC TÍCH HỢP

7.1. Quy ước dữ liệu bàn cờ

- 1: quân đi trước
- -1: quân đi sau
- 0: ô trống

7.2. Interface chuẩn

```
struct AIMove {
    int row;

    int col;

    long score;

    long nodes_evaluated;

    float time_ms;

    bool is_timeout;
};
```

```
AIMove get_best_move(int board[15][15], int player_turn, int max_depth,
    int timeout_ms, bool use_alpha_beta);
```

7.3. Ý nghĩa tham số

- board[15][15]: trạng thái hiện tại của bàn cờ
- player_turn: phe mà AI đang tính nước đi tốt nhất cho lượt này, nhận giá trị 1 hoặc -1
- max_depth: độ sâu tìm kiếm tối đa mong muốn
- timeout_ms: giới hạn thời gian cho một lượt suy nghĩ
- use_alpha_beta: bật hoặc tắt Alpha-Beta để benchmark đối chứng

7.4. Yêu cầu bắt buộc

- không được sửa chữ ký hàm nếu chưa thống nhất toàn nhóm
- frontend và engine phải dùng đúng một chuẩn dữ liệu duy nhất
- không hardcode AI vào một phe cố định

8. VAI TRÒ KỸ THUẬT THAM KHẢO

Lưu ý:

Mục này chỉ mô tả vai trò kỹ thuật tham khảo.

Khi giao việc cụ thể theo thành viên, phải dùng **bảng phân công công việc (WBS) chính thức**.

8.1. AI Architect & Core Algorithm

Phạm vi tham khảo

- cài đặt Minimax
- cài đặt Alpha-Beta Pruning
- cài đặt Candidate Move Generation
- cài đặt Iterative Deepening
- tích hợp Heuristic vào AI Core

Chuẩn đầu ra tham khảo

- Alpha-Beta giảm $\geq 60\%$ số node so với Minimax thuần
- phản hồi ổn định < 2 giây tại Depth 4
- không crash hoặc trả về nước đi rỗng khi timeout

8.2. Backend & Engine Runtime

Phạm vi tham khảo

- quản lý mảng trạng thái bàn cờ 15x15
- kiểm tra nước đi hợp lệ
- kiểm tra thắng/hòa bằng local check
- xử lý invalid move
- tích hợp WASM vào Web Worker
- đảm bảo đồng bộ dữ liệu giữa UI và AI

Chuẩn đầu ra tham khảo

- vượt qua 100% test luật
- nhận diện đúng thắng, hòa, invalid move
- không có lỗi sai lệch trạng thái board giữa JS và WASM trong demo

8.3. Heuristic Analyst & QA Presenter

Phạm vi tham khảo

- xây dựng bảng trọng số Heuristic
- chuẩn hóa test format
- soạn bộ 15 Scenario Tests
- xây dựng slide giải thích logic AI

Phân loại Heuristic bắt buộc

- Open Four
- Half-Open Four
- Open Three
- Half-Open Three
- Open Two

Ràng buộc Heuristic

- Open Four > Half-Open Four > Open Three > Half-Open Three > Open Two
- điểm phòng thủ với nguy cơ thua ngay phải cao hơn điểm tấn công trung bình

Chuẩn đầu ra tham khảo

- AI pass $\geq 90\%$ trên bộ 15 Scenario Tests
- slide có giải thích rõ logic score và ví dụ minh họa

8.4. Experiment & Benchmark Data

Phạm vi tham khảo

- thiết lập hệ thống logger
- thu thập dữ liệu node, time, score, depth
- benchmark Minimax và Alpha-Beta
- vẽ biểu đồ phục vụ báo cáo

Chuẩn đầu ra tham khảo

- benchmark tại các depth 3, 4, 5
- dữ liệu benchmark sạch và tái hiện được
- benchmark công bằng theo cùng board state, depth, move generation, move order và build mode

8.5. Frontend & UX

Phạm vi tham khảo

- dựng giao diện bàn cờ 15x15
- xử lý click chuột
- hiển thị trạng thái game
- hiển thị metrics thời gian thực
- xây loading spinner, dashboard metrics, control panel
- đảm bảo UI không block Main Thread

Chuẩn đầu ra tham khảo

- click đặt quân phản hồi ngay, không lag
- có loading spinner khi AI đang suy nghĩ
- highlight rõ ô vừa đánh
- có control panel và dashboard
- reset xóa sạch board, spinner và metrics

9. ĐỊNH DẠNG KIỂM THỬ TỰ ĐỘNG

9.1. Format chuẩn của 1 test case

File JSON hoặc CSV chuẩn có cấu trúc:

- Test_ID
- Board_State
- Player_Turn
- Acceptable_Moves
- Expected_Priority
- Pass/Fail

9.2. Ý nghĩa

- **Board_State**: ma trận 15x15 đầu vào
- **Player_Turn**: phe mà AI đang tính cho
- **Acceptable_Moves**: danh sách các tọa độ được chấp nhận là đúng
- **Expected_Priority**: mô tả ngắn logic chiến thuật mong đợi
- **Pass/Fail**: kết quả khi so đầu ra AI với **Acceptable_Moves**

9.3. Cấu trúc bộ 15 tests đề xuất

Nhóm 1 – Luật & Engine (5 case)

- kiểm tra thắng ngang
- kiểm tra thắng dọc
- kiểm tra thắng chéo
- invalid move
- hòa khi bàn đầy mà không ai thắng

Nhóm 2 – Sinh tử chiến thuật (5 case)

- chặn Half-Open Four của đối thủ
- tự đánh nước dứt điểm thắng ngay
- ưu tiên chặn nguy cơ thua ngay
- không bỏ lỡ nước thắng tức thì
- chọn nước tạo thế mạnh hơn giữa 2 ứng viên

Nhóm 3 – Chiến thuật nâng cao (5 case)

- ưu tiên Open Four hơn Open Three
- phát hiện song công
- xử lý thế cờ giữa bàn
- thắng ở ô thứ 225 cuối cùng
- tình huống có nhiều nước đúng tương đương

bảng phân công công việc

Nhánh làm việc trên git

Thành viên 1 dùng **ai-core**

Thành viên 2 dùng **engine-runtime**

Thành viên 3 dùng **heuristic-qa**

Thành viên 4 dùng **benchmark-logger**

Thành viên 5 dùng **frontend-ui**

dùng đúng nhánh git của mình đừng dùng sai nhánh nhé

BẢNG PHÂN CHIA CÔNG VIỆC

Dự án: AI Caro Engine

Thành viên	Vai trò Kỹ thuật	Nhiệm vụ chính (Focus)	KPI Kỹ thuật
Thành viên 1	AI Architect	Xây dựng thuật toán Minimax, Alpha-Beta, Iterative Deepening.	Alpha-Beta cắt tỉa >= 60% Node. Phản hồi < 2s (Depth 4).
Thành viên 2	System Integration	Quản lý Game Engine, Luật thắng/hòa, Tích hợp WASM & Web Worker.	Pass 100% test luật. Hoàn thành WASM Spike trong 3 ngày.
Thành viên 3	Strategy Analyst	Thiết kế Heuristic Weight Table, Soạn 15 bộ Scenario Tests.	AI pass >=90% bộ test. Pass 100% nhóm luật & sinh tử.
Thành viên 4	Data Engineer	Xây dựng Logger, thực hiện Benchmark, thu thập số liệu thực nghiệm.	Đủ dữ liệu Benchmark Depth 3, 4, 5. Số liệu sạch & tái hiện được.

Thành viên 5	UI/UX Developer	Vẽ Board, Dashboard Metrics, xử lý Non-blocking UI & Control Panel.	Click phản hồi ngay. Reset xóa sạch Board & Metrics.
-------------------------	----------------------------	--	---

công việc chi tiết

bảng phân công công việc chi tiết, theo kiểu mỗi thành viên phải làm gì, làm đến mức nào mới được tính là hoàn thành, bám sát đúng project AI Caro Engine (Minimax & Alpha-Beta) mà bạn đã chốt.

BẢNG PHÂN CÔNG CÔNG VIỆC CHI TIẾT

Dự án: AI Caro Engine

Công nghệ: C++ + WASM + HTML/CSS/JS

Phạm vi: Bàn cờ 15x15, thắng khi có từ 5 quân liên tiếp trở lên, hòa khi đầy bàn mà không ai thắng

1. Thành viên 1 – AI Architect & Algorithm Lead

Vai trò chính

Phụ trách toàn bộ **lỗi AI** của hệ thống. Đây là người làm phần quan trọng nhất: để AI biết suy nghĩ, chọn nước đi và phản hồi trong thời gian hợp lý.

Công việc phải làm

1. Cài đặt thuật toán Minimax

Phải viết được thuật toán Minimax để:

- nhận trạng thái bàn cờ hiện tại
- sinh ra các nước đi hợp lệ
- mô phỏng các lượt đi tiếp theo
- tính toán nước đi tốt nhất cho AI

2. Cài đặt Alpha-Beta Pruning

Phải tích hợp Alpha-Beta vào Minimax để:

- cắt bỏ các nhánh không cần xét
- giảm số node duyệt
- tăng tốc độ phản hồi

3. Cài đặt Candidate Move Generation

Phải viết bộ sinh nước đi theo đúng quy tắc:

- không xét toàn bộ 225 ô
- chỉ xét ô trống có ít nhất một quân bất kỳ trong vùng **Chebyshev distance ≤ 2**
- nếu bàn cờ trống hoàn toàn thì chọn ô trung tâm **(7, 7)** làm ứng viên đầu tiên

4. Cài đặt Iterative Deepening

Phải làm cho AI tìm kiếm theo từng mức depth:

- depth 1
- depth 2
- depth 3
- ...
- đến depth tối đa hoặc đến khi hết thời gian

5. Cài đặt timeout 2000ms

Phải đảm bảo:

- AI không được treo quá 2 giây
- nếu hết thời gian giữa chừng thì trả về nước đi tốt nhất ở depth hoàn chỉnh gần nhất
- không được crash
- không được trả về nước đi rỗng

6. Tích hợp Heuristic

Phải nhận bảng điểm từ Thành viên 3 và tích hợp vào AI để:

- chấm điểm thế cờ
- so sánh nước đi
- chọn nước đi “khôn” hơn

7. Hỗ trợ xuất dữ liệu benchmark

Phải phối hợp với Thành viên 4 để logger lấy được:

- số node duyệt
- thời gian chạy
- score heuristic
- depth đạt được
- trạng thái timeout

Sản phẩm phải bàn giao

- Hàm **get_best_move(...)** chạy đúng
- Minimax hoạt động
- Alpha-Beta hoạt động
- Candidate Moves hoạt động đúng chuẩn
- Iterative Deepening hoạt động

- Timeout 2000ms hoạt động ổn định
- AI đánh được bằng heuristic đã tích hợp

Điều kiện được tính là hoàn thành

- AI phản hồi ổn định < 2 giây ở Depth 4
- Alpha-Beta giảm $\geq 60\%$ số node so với Minimax thuần trong benchmark công bằng
- Không crash, không trả về null, không treo khi timeout
- Ghép được với engine và UI

2. Thành viên 2 – Backend & Engine Runtime

Vai trò chính

Phụ trách **Game Engine và tích hợp hệ thống**. Đây là người đảm bảo game chạy đúng luật và AI kết nối được với giao diện.

Công việc phải làm

1. Xây dựng board 15x15

Phải quản lý được:

- trạng thái bàn cờ `board[15][15]`
- ô trống, ô có quân
- reset game
- đổi lượt người chơi / AI

2. Xử lý invalid move

Phải đảm bảo:

- không cho đánh vào ô đã có quân
- không cho cập nhật sai trạng thái bàn cờ
- nếu người dùng click sai thì game không bị lỗi

3. Cài đặt kiểm tra thắng

Phải viết hàm kiểm tra thắng bằng **local check**:

- chỉ quét từ ô vừa đánh
- quét 4 hướng:
 - ngang
 - dọc
 - chéo chính
 - chéo phụ
- nếu tổng quân cùng màu trên một trục ≥ 5 thì báo thắng ngay

4. Cài đặt kiểm tra hòa

Phải đảm bảo:

- nếu chưa có ai thắng
- và số ô đã đánh = 225
- thì báo hòa

5. Quản lý game loop

Phải làm được luồng:

- người chơi đánh
- kiểm tra thắng/hòa
- nếu chưa kết thúc thì gọi AI
- AI đánh
- kiểm tra thắng/hòa
- tiếp tục hoặc kết thúc

6. Tích hợp WASM

Phải:

- biên dịch lỗi C++ sang WASM bằng Emscripten
- gọi được hàm AI từ JavaScript
- đảm bảo dữ liệu board truyền đúng

7. Thiết lập Web Worker

Phải chạy AI trong Web Worker để:

- không block giao diện
- tránh đơ UI khi AI đang suy nghĩ

8. Thực hiện WASM Spike

Trong 3 ngày đầu phải kiểm tra:

- có build được WASM không
- UI có gọi được hàm AI không
- worker có chạy ổn định không

9. Báo động fallback nếu cần

Nếu WASM fail theo tiêu chí đã chốt thì phải:

- báo ngay cho nhóm
- cung cấp tình trạng cụ thể
- phối hợp chuyển sang backend local nếu cần

Sản phẩm phải bàn giao

- Game Engine hoàn chỉnh
- Logic thắng/hòa đúng
- Invalid move đúng
- WASM chạy được hoặc có báo cáo fail hợp lệ
- Worker chạy ổn định
- Board đồng bộ đúng giữa JS và AI

Điều kiện được tính là hoàn thành

- Pass **100% bộ test luật**
 - thắng
 - hòa
 - invalid move
- Không có lỗi lệch trạng thái board giữa JS và WASM
- Hoàn thành WASM Spike trong 3 ngày đầu
- Nếu fail phải báo đúng lúc, không để kéo dài âm thầm

3. Thành viên 3 – Heuristic Analyst & QA Presenter

Vai trò chính

Phụ trách **chiến thuật đánh giá thế cờ**, **bộ test tình huống**, và phần **giải thích học thuật khi bảo vệ**.

Công việc phải làm

1. Xây dựng Heuristic Weight Table

Phải định nghĩa điểm số cho các mẫu:

- Open Four
- Half-Open Four
- Open Three
- Half-Open Three
- Open Two

2. Chốt thứ bậc trọng số

Phải đảm bảo bảng điểm không mâu thuẫn, ít nhất theo thứ tự:

- Open Four > Half-Open Four > Open Three > Half-Open Three > Open Two

3. Thiết kế nguyên tắc ưu tiên phòng thủ

Phải đảm bảo:

- nếu đối thủ có nguy cơ thắng ngay, AI phải ưu tiên chặn
- điểm chặn nguy cơ thua ngay phải lớn hơn điểm tấn công trung bình
- ví dụ: chặn Half-Open Four của đối thủ phải quan trọng hơn tạo Open Three cho mình

4. Soạn bộ 15 Scenario Tests

Phải tạo file test chuẩn gồm:

- Test_ID
- Board_State
- Player_Turn
- Acceptable_Moves
- Expected_Priority
- Pass/Fail

5. Chia test thành 3 nhóm

Phải có đủ:

- **Nhóm luật & engine**
 - thắng ngang/dọc/chéo
 - hòa
 - invalid move
- **Nhóm sinh tử chiến thuật**
 - chặn nước thua ngay
 - không bỏ lỡ nước thắng ngay
- **Nhóm chiến thuật nâng cao**
 - ưu tiên thế mạnh hơn
 - song công
 - nhiều nước đúng tương đương

6. Tuning heuristic với Thành viên 1

Phải:

- chạy test
- xem AI chọn nước gì
- nếu AI chọn chưa hợp lý thì điều chỉnh trọng số
- lặp lại cho đến khi đạt KPI

7. Chuẩn bị slide thuyết trình

Phải viết phần:

- heuristic là gì
- vì sao chấm điểm như vậy
- Alpha-Beta giúp gì
- vì sao AI chọn một nước cụ thể

Sản phẩm phải bàn giao

- Weight Table hoàn chỉnh
- File 15 Scenario Tests
- Ghi chú logic cho từng test
- Slide phần heuristic và AI reasoning

Điều kiện được tính là hoàn thành

- AI pass $\geq 90\%$ toàn bộ 15 tests
- Nhóm test luật và sinh tử phải pass **100%** khi tích hợp hoàn chỉnh
- Slide phải giải thích được rõ logic score và ví dụ cụ thể

4. Thành viên 4 – Experiment & Benchmark Data

Vai trò chính

Phụ trách **ghi log, benchmark, biểu đồ và số liệu thực nghiệm** để chứng minh dự án có giá trị học thuật.

Công việc phải làm

1. Thiết lập logger

Phải thu được:

- số node đã duyệt
- thời gian chạy
- score heuristic
- depth đạt được
- timeout có xảy ra hay không

2. Thiết kế format log

Phải thống nhất format rõ ràng để:

- dễ đọc
- dễ đối chiếu
- dễ chuyển thành biểu đồ

3. Benchmark Minimax vs Alpha-Beta

Phải so sánh trực tiếp:

- Minimax thuần
- Minimax + Alpha-Beta

4. Đảm bảo benchmark công bằng

Phải chạy trong điều kiện:

- cùng trạng thái bàn cờ
- cùng depth
- cùng move generation
- cùng thứ tự duyệt nhánh
- cùng build mode
- ưu tiên Release mode

5. Chạy benchmark ở các mức depth

Ít nhất phải có:

- Depth 3
- Depth 4
- Depth 5

6. Ghi rõ cấu hình máy test

Phải lưu:

- CPU
- RAM
- hệ điều hành
- trình duyệt
- build mode

7. Làm biểu đồ

Phải biến số liệu thành:

- bảng benchmark
- biểu đồ cột
- biểu đồ đường nếu cần

8. Bàn giao dữ liệu cho TV3

Phải chuyển biểu đồ và dữ liệu cho Thành viên 3 để đưa vào slide bảo vệ

Sản phẩm phải bàn giao

- Logger hoàn chỉnh
- File benchmark
- Bảng số liệu
- Biểu đồ thực nghiệm
- Thông tin cấu hình máy test

Điều kiện được tính là hoàn thành

- Có đủ benchmark cho Depth 3, 4, 5

- Dữ liệu sạch, tái hiện được
- Biểu đồ chứng minh rõ Alpha-Beta giảm node và giảm thời gian

5. Thành viên 5 – Frontend & UX Developer

Vai trò chính

Phụ trách **giao diện game** và toàn bộ phần **trình diễn trực tiếp**.

Công việc phải làm

1. Dựng bàn cờ 15x15

Phải hiển thị được:

- lưới bàn cờ
- quân cờ X/O hoặc đen/trắng
- vị trí các quân đã đánh

2. Highlight nước đi mới nhất

Phải làm nổi bật ô vừa đánh để:

- người xem dễ theo dõi
- demo rõ ràng hơn

3. Hiển thị trạng thái lượt chơi

Phải có hiển thị:

- đang tới lượt ai
- AI đang suy nghĩ hay chưa
- game đã kết thúc hay chưa

4. Xây Dashboard Metrics

Phải hiển thị:

- Nodes Evaluated
- Time (ms)
- Score
- Depth hiện tại nếu có

5. Xây Control Panel

Phải có:

- chọn độ sâu
- chọn Minimax / Alpha-Beta

- reset game

6. Loading Spinner

Phải hiện spinner khi AI đang chạy

7. Không block Main Thread

Phải đảm bảo:

- UI không bị đơ
- vẫn phản hồi bình thường khi AI suy nghĩ

8. Khóa click khi AI đang nghĩ

Phải:

- không cho người chơi click thêm
- tránh lỗi state

9. Reset game hoàn chỉnh

Nút reset phải:

- xóa bàn cờ
- xóa highlight
- tắt spinner
- reset metrics về 0

Sản phẩm phải bàn giao

- giao diện game hoàn chỉnh
- dashboard metrics hoạt động
- control panel hoạt động
- loading spinner hoạt động
- reset hoạt động đúng

Điều kiện được tính là hoàn thành

- Click phản hồi ngay, không lag
- UI không block Main Thread
- Reset xóa sạch board và metrics
- Giao diện đủ rõ để demo trước hội đồng

6. GIAO THỨC PHỐI HỢP CHUNG

Interface AI

`get_best_move(board, player_turn, max_depth, timeout_ms, use_alpha_beta)`

Quy ước board

- 1 = quân đi trước
- -1 = quân đi sau
- 0 = ô trống

Nguyên tắc

- không tự ý sửa interface
- không tự ý sửa luật
- không tự ý sửa format test
- có blocker kỹ thuật lớn phải báo cho nhóm